All the Emojis! (`morse`)

William and Edoardo are using a new instant messaging app to communicate. However, the application only supports messages composed exclusively of lowercase letters of the English alphabet. They are avid emoji users, so this is clearly unacceptable!

For this reason, they devised a way to convert any message to a string that can be sent using the app: any symbol in the original message is mapped to a sequence of one or more lowercase letters and this encoding is known to both of them. The encoding is defined for M symbols, mapping the i-th of which to a sequence A_i, composed only of lowercase letters.

Edoardo has received a message S, composed of N lowercase letters, and now needs to translate it back to its original form. However, he realized that, in some cases, a message can be interpreted in multiple ways. This is a huge problem! For instance, consider the following encoding, where $M = 7$:

`Y` → `a` `e` → `ab` `s` → `bc` `!` → `c` `N` → `aa` `o` → `bb` `?` → `cc`

If the message is `Yes!`, William will actually send to Edoardo `aabbcc`. However, also the message `No?` is translated into `aabbcc`. What a mess! Help Edoardo count the number of different ways in which the message S can be interpreted.

> ☞ Among the attachments of this task you may find a template file `morse.*` with a sample incomplete implementation.

Input

The first line contains two integers N and M, the number of lowercase letters in the received message and the number of different symbols in their encoding. The second line contains a string S composed of N lowercase letters of the English alphabet, the message received by Edoardo. Further M lines follow, the i-th of which containing a string A_i composed of lowercase letters, the mapping of the i-th symbol in the encoding.

Output

You need to write a single line with an integer: the number of different ways in which the received message can be interpreted. Since this number can be very big, calculate it modulo $10^9 + 7$.

> ✍ The *modulo* operation ($a \bmod m$) can be written in C++ as `(a % m)`. To avoid the *integer overflow* error, remember to reduce all partial results through the modulus, and not just the final result!
> *Notice that if $x < 10^9 + 7$, then $2x$ fits into a* C++ `int`.

Constraints

- $1 \le N \le 4000$.
- $1 \le M \le 4000$.
- $1 \le |A_i| \le N$, for each $i = 0 \dots M-1$.
- $A_i \neq A_j$ for each $i \neq j$.

Scoring

Your program will be tested against several test cases grouped in subtasks. In order to obtain the score of a subtask, your program needs to correctly solve all of its test cases.

– **Subtask 1** (0 points) Examples.

– **Subtask 2** (15 points) $N \le 20$, $M \le 2$.

– **Subtask 3** (12 points) $|A_i| = 1$ for each $i = 0 \dots M-1$.

– **Subtask 4** (18 points) $N \le 500$, $M \le 500$.

– **Subtask 5** (24 points) S and A_i are composed only of the letter `'a'`, for each $i = 0 \dots M-1$.

– **Subtask 6** (31 points) No additional limitations.

Examples

input	output

`4 4`
`aabb`
`a`
`b`
`bb`
`ab`
	`3`

`4 4`
`aabb`
`a`
`ab`
`aa`
`aba`
	`0`

Explanation

In the **first sample case**, the message `aabb` can be interpreted in three ways:

`a|ab|b   a|a|bb   a|a|b|b`

In the **second sample case**, the message cannot be interpreted in any way using the given mapping!